1.1 Programmiersprachen

Programmiersprachen werden in historischer Sicht in fünf sog. *Programmiersprachengenerationen* eingeteilt:

	1. *Maschinensprache(n)*	2. *Assembler-Sprachen*	3. *Prozedurale Sprachen*	4. *4 GL (Generation Language)*	5. *Künstliche Intelligenz*
Beschreibung	Eingabe der Anweisungen als *Bitfolge* d. h. 0 und 1	werden durch *Assembler* in Maschinencode umgewandelt, der wie Maschinensprache an einen bestimmten Prozessor gebunden ist	sind unabhängig vom *Computersystem* und werden auch als *höhere* Programmiersprachen bezeichnet. Das notwendige *Übersetzungsprogramm* (z. B. *Interpreter, Compiler*) erzeugt den Maschinencode. Ihre Struktur und Befehlsvorrat sind auf bestimmte Anwendungsbereiche zugeschnitten.	Programmierer beschreibt, was das Programm leisten soll, ohne den genauen Algorithmus anzugeben. Einzelne Anweisung löst Folge von internen Einzelschritten aus. sind auf spezielle Anwendungsgebiete ausgerichtet, z. B. Bearbeiten und Auswerten von Dateien, Datenbanken, Tabellenkalkulationen oder auf Erstellen von Bildschirmformularen	Programmiersprachen, die entwickelt wurden, um zu erforschen, unter welchen Bedingungen Computer *intelligente menschliche Verhaltensweisen* nachvollziehen können. Sie gehören meist zu den *logischen oder funktionalen* Programmiersprachen.
Vorteile	*(schnell, da keine Übersetzung nötig)*	besser *lesbar* als Maschinensprache; schnellerer *Programmablauf (als Gen. 3)* bei geringem *Speicherplatzbedarf*	leichter *erlernbar*; auch bei anderen *Rechnersystemen* verwendbar	leichte Programmierung durch komfortable und leistungsfähige *Entwicklungssysteme*	
Nachteile	▪ hoher *Programmieraufwand* ▪ schlechte *Lesbarkeit* ▪ muss für jeden Computer *neu entwickelt* werden		(früher) im Vergleich zu Maschinenprogrammen höherer *Speicherplatzbedarf* und langsamerer *Programmablauf* (wird aber durch optimierte Übersetzung wieder ausgeglichen)	*Geringer Einfluss* des Programmierers auf interne Abläufe, *langsamerer Programmablauf* als bei Generation 3	
Beispiele (Sprachen)			Cobol, RPG, Fortran, Pascal, PL/1, Basic, Ada, C/C++, Java, uvm.	SQL, Natural, Symphony, Open Access	Prolog, Lisp, Smalltalk

Darüber hinaus werden Programmiersprachen in *Sprachtypen* eingeteilt:

Prozedurale Programmiersprachen	*Objektorientierte* Programmiersprachen	*Skriptsprachen und hybride* Programmiersprachen	*Funktionale und logische* Programmiersprachen
…geben den Weg zur Problemlösung als *Folge von Einzelschritten* an, die *nacheinander abgearbeitet* werden Beispiele: *BASIC*, ursprünglich für Schulungszwecke entwickelt, weiterentwickelt u. a. zu Visual Basic, ab 2002 im .NET-Framework integriert *FORTRAN*, v. a. für mathematische oder technische Anwendungen *C*, v. a. für Entwicklung von systemnahen Programmen, wie z. B. UNIX *PASCAL*, ursprünglich entwickelt als Hilfsmittel zur Erlernung der Programmierung, weiterentwickelt zu TURBO-PASCAL und DELPHI. *COBOL*, v. a. für betriebswirtschaftliche Anwendungen Sprachen dieses Typs werden auch als *imperative Sprachen* bezeichnet.	…entstanden aus der Erkenntnis, dass in der prozeduralen Programmierung eine Verbindung zwischen *den Daten und den sie verarbeitenden Funktionen* fehlt. Idee: Ausschnitte aus der Realität werden durch Objekte abgebildet. Die Modellierung orientiert sich nicht mehr vorrangig an Vorgängen wie Bestellung erfassen oder Betrag berechnen, sondern an den jeweiligen Objekten wie z. B. Konto, Waren. Ein Objekt ist eine Kombination von Daten und Funktionen (Methoden), die miteinander über eine Schnittstelle in Beziehung stehen. Die Trennung von Daten und Funktion wird also aufgehoben, der Zugriff auf die Daten wird nur über die Schnittstelle erlaubt. Beispiele: *C++*, geeignet für systemnahe Programme und komplexe Anwendungen *Java*, eine schlanke und übersichtliche Sprache *C#*, als Weiterentwicklung von Java, C++, C und Delphi; bequeme Erstellung von Windows-Programmen	Hybride Sprachen sind Sprachen, die sowohl *prozedural als auch objektorientiert* eingesetzt werden können. Skriptsprachen sind meist *einfacher Natur* und werden meist durch einen *Interpreter* (zur Laufzeit) übersetzt. Programme, die in Skriptsprachen geschrieben sind, werden *Skripte* oder *Makros* genannt. Beispiele: *JavaScript*, ursprünglich für Erweiterung von HTML in Webbrowsern entwickelt, um Benutzerinteraktionen auszuwerten und dynamisch Inhalte einer Webseite zu verändern. (Microsoft-Dialekt: JScript) *PHP*, sehr verbreitete Sprache bei serverseitiger Webprogrammierung	Funktional: Das Programm ist eine Funktion, die sich auf einfachere Funktionen stützt. Eine Funktion kann eine andere Funktion aufrufen, oder das Ergebnis einer Funktion kann als Parameter für eine andere Funktion genutzt werden. Programme werden wie mathematische Funktionen mit Definitions- und Wertebereich geschrieben. Aus Eingabewerten wird der Wert der Funktion berechnet. Beispiel: *LISP* Logisch: Zur Lösung des Problems wird kein Algorithmus angegeben, sondern nur die Bedingungen für eine korrekte Lösung bestimmt. Beispiel: *PROLOG* Beide Sprachtypen gemeinsam werden auch als *deklarative Sprachen* bezeichnet.